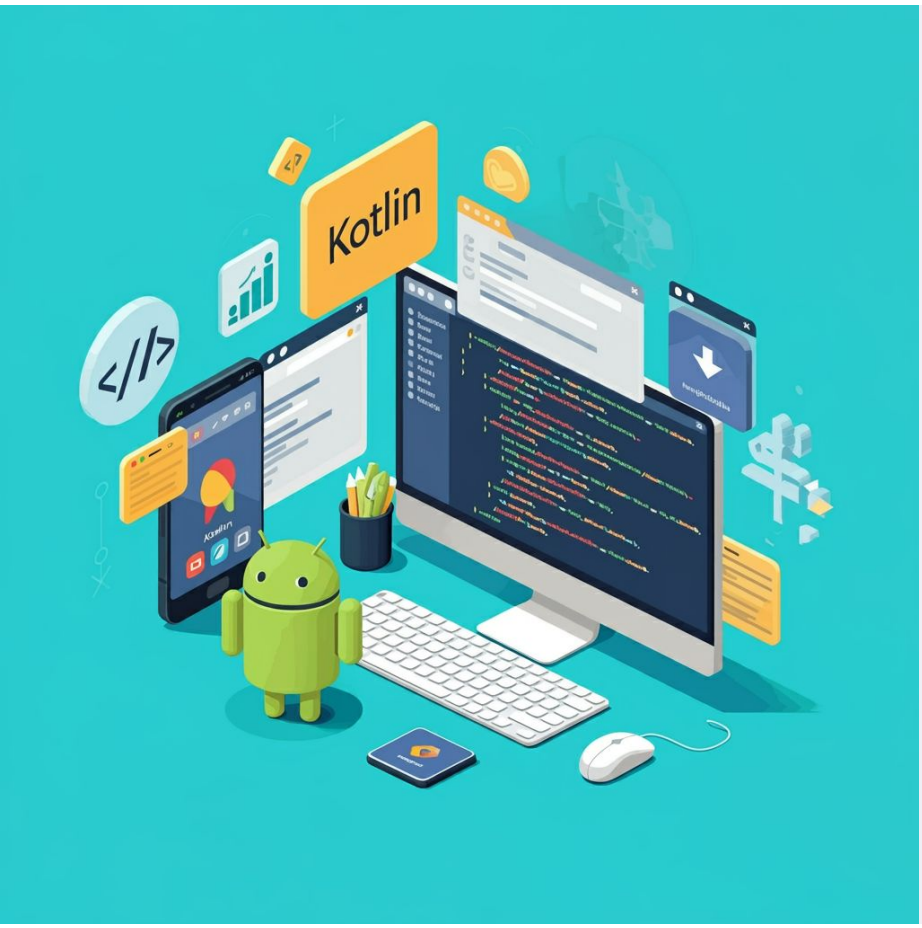
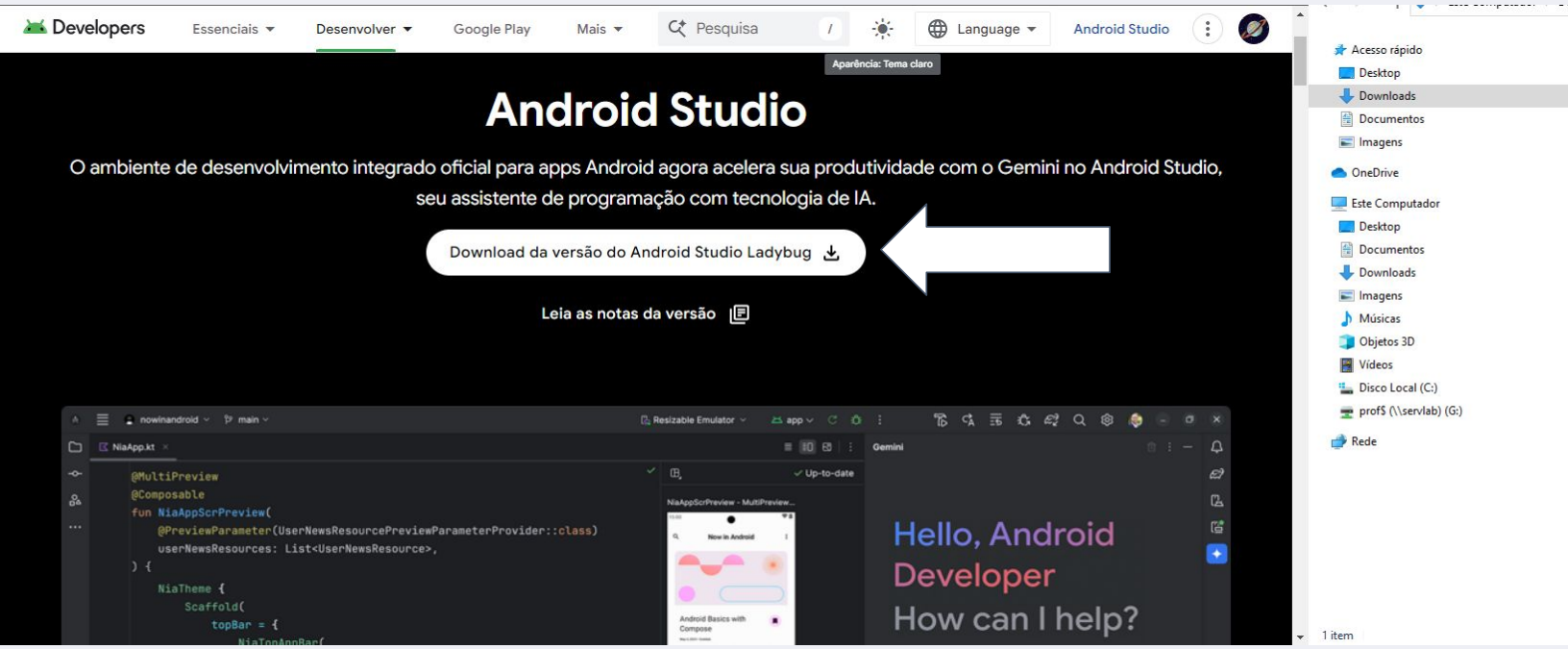


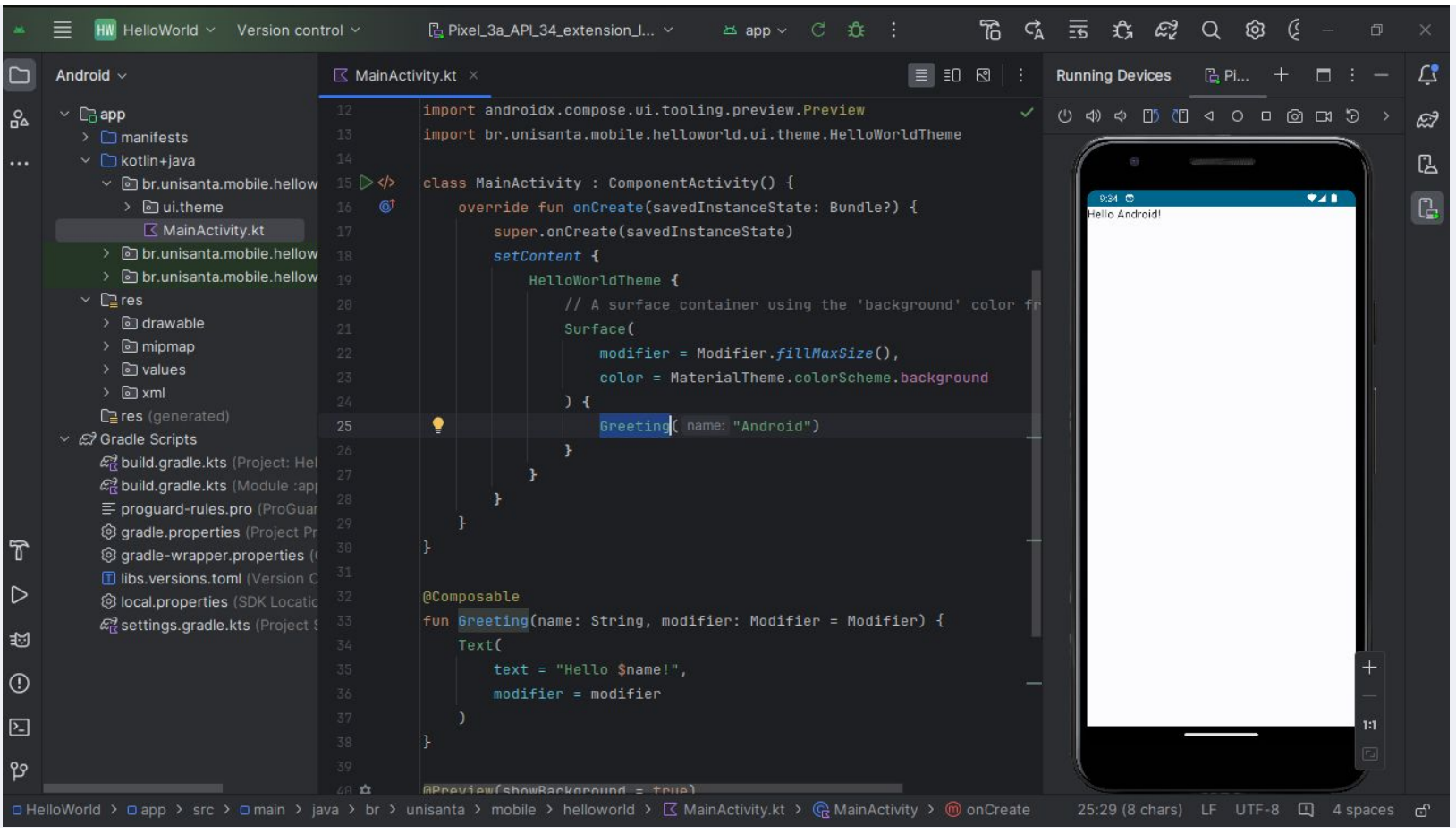
Aula 2: Mergulhando no Android com Android Studio



Android Studio

Android Studio, a IDE oficial para desenvolvimento Android. Junto ao Android Studio, está o Android SDK, o emulador e tudo mais que for necessário para o desenvolvimento.


Developers
Essenciais
Desenvolver
Google Play
Mais
Pesquisa
Language
Android Studio
Aparência: Tema claro
Android Studio
O ambiente de desenvolvimento integrado oficial para apps Android agora acelera sua produtividade com o Gemini no Android Studio, seu assistente de programação com tecnologia de IA.
Download da versão do Android Studio Ladybug
Leia as notas da versão
Hello, Android Developer
How can I help?
Acesso rápido
Desktop
Downloads
Documentos
Imagens
OneDrive
Este Computador
Desktop
Documentos
Downloads
Imagens
Músicas
Objetos 3D
Vídeos
Disco Local (C:)
Rede
1 item


HelloWorld
Version control
Pixel_3a_API_34_extension_l...
app
Android
MainActivity.kt
Running Devices
import androidx.compose.ui.tooling.preview.Preview
import br.unisanta.mobile.helloworld.ui.theme.HelloWorldTheme
class MainActivity : ComponentActivity() {
override fun onCreate(savedInstanceState: Bundle?) {
super.onCreate(savedInstanceState)
setContent {
HelloWorldTheme {
Surface(
modifier = Modifier.fillMaxSize(),
color = MaterialTheme.colorScheme.background
) {
Greeting(name: "Android")
}
}
}
}
}
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
Text(
text = "Hello $name!",
modifier = modifier
)
}
Hello Android!
HelloWorld › app › src › main › java › br › unisanta › mobile › helloworld › MainActivity.kt › MainActivity › onCreate
25:29 (8 chars)
LF
UTF-8
4 spaces

O QUE É O ANDROID SDK?

Podemos dizer que o Android SDK é nosso kit de desenvolvimento. A sigla SDK significa Software Development Kit. O SDK do Android vem com todas as ferramentas, APIs e bibliotecas necessárias para se trabalhar com a plataforma Android. Junto ao SDK também estão os emuladores, que você utilizará para testar seus aplicativos. Não se preocupe em instalá-lo, pois ele vem junto com o Android Studio.



REQUISITOS DO SISTEMA

1 Windows

- Microsoft® Windows® 8/10 64 bits
- Mínimo de 8 GB de RAM
- Espaço livre em disco 20 GB

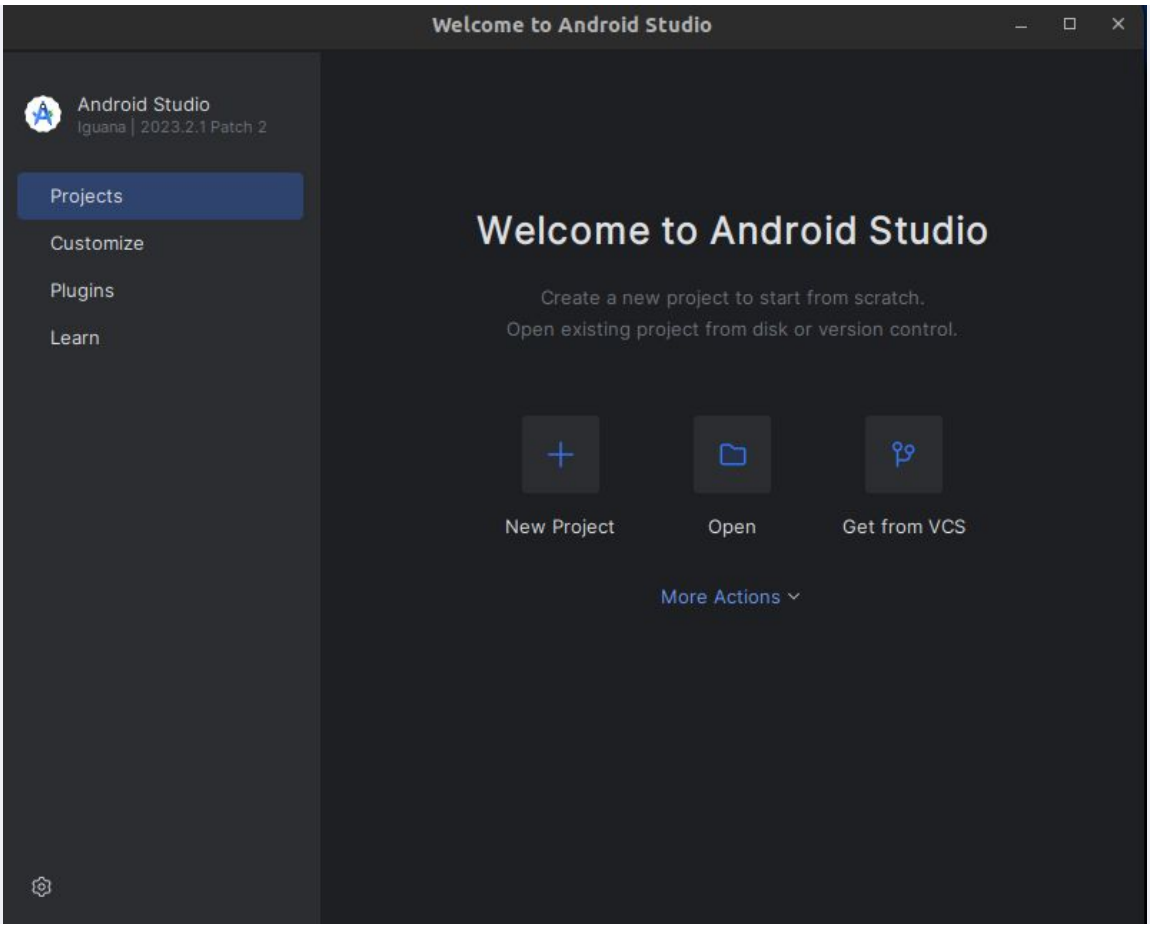
2 Mac

- MacOS® X 10.10 (Yosemite) ou posterior
- Mínimo de 8 GB de RAM
- Espaço livre em disco 20 GB

3 Linux

- Área de trabalho GNOME ou KDE;
- Mínimo de 8 GB de RAM
- Espaço livre em disco 20 GB

SSD e Placa de Vídeo Ajuda no Desempenho!



CONFIGURANDO O SDK

Possuir o SDK é um pré-requisito para que tudo funcione corretamente. O Android Studio já instala a última versão disponível durante a instalação do programa, então já está tudo configurado corretamente para começarmos a programar.



PRIMEIRO PROJETO: HELLO WORLD



EXPLICAÇÃO APP PADRÃO

- Em Android, uma **Activity** significa basicamente uma tela; é o ponto central no qual o usuário interage com o App.
- Package e Reverse Domain Name
- Em Save Location , você deve indicar a pasta para salvar o projeto.
- Minimum SDK API 27 = Android 7.0
- Repare que existe uma pasta chamada app e uma chamada Gradle Scripts .



GRADLE

O Gradle é uma ferramenta que automatiza a compilação de um software. O Android Studio utiliza o Gradle para compilação e também como gerenciador de dependências.



GRADLE

O arquivo **build.gradle do projeto** contém configurações relacionadas ao projeto todo, como as configurações de **repositórios** de **dependências**, versão do **plug-in** do Gradle e versão do Kotlin. Na prática, dificilmente mexemos nessas configurações.



GRADLE

Já o arquivo **build.gradle do módulo** contém configurações referentes ao aplicativo em si, como **versão** mínima do **sistema**, versão de **compilação** e também as **bibliotecas** necessárias.

Por Agora não vamos modificar nada neste arquivo também.



MAINACTIVITY

Observe que existe uma pasta **java** e, dentro dela, uma pasta **br.com.instituicao.helloworld** e um arquivo chamado **MainActivity.**

É nessa pasta que criaremos todos os arquivos em Kotlin – é aqui que fica todo o código do nosso aplicativo!



RES

Um pouco mais abaixo existe uma pasta chamada **res** , que é uma abreviação de **resources** (recursos). É lá que ficam todos os arquivos de **layout, imagens** que usaremos nos projetos



RES - COMPONENTIZAÇÃO

layout : é onde colocaremos todos os arquivos xml relacionados ao layout do App. Repare que já existe um arquivo chamado **activity_main.xml** . Ele foi criado pelo Android Studio quando selecionamos o modelo "Empty Activity". Esse arquivo contém o código em xml responsável pela criação da tela do nosso app;



RES - VALUES\COLORS.XML

values : nessa pasta ficam os arquivos xml de colors, strings e styles; colors.xml : nesse arquivo podemos configurar cores para usar no nosso projeto. Imagine que você defina que a cor dos botões do seu App será padronizada com o tom de azul #4169E1 .

<color name="button_color">#4169E1</color>

@colors/button_color ;



RES - VALUES\STRING.XML

String: Neste arquivo podemos configurar textos que serão usados no nosso App.

<string name="mensagem_erro">Ocorreu um erro</string>

Depois simplesmente referenciá-la pelo nome:

@strings/mensagem_erro ;

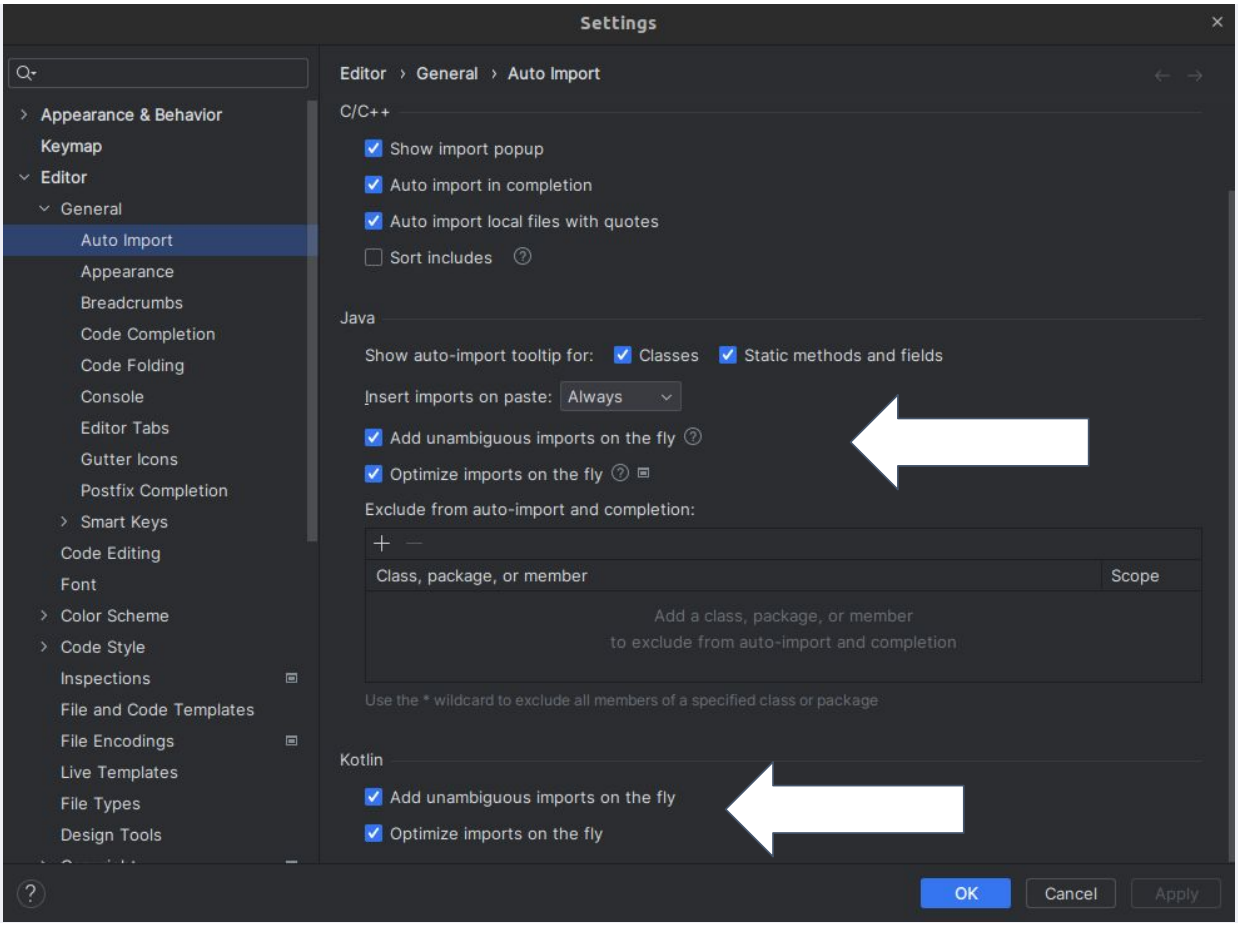
Configuração de uso global!



EMULADOR ANDROID

O emulador virtualiza o sistema Android; dessa forma, conseguimos testar o aplicativo sem a necessidade de ter um aparelho.

Podemos criar vários emuladores virtualizando modelos de celulares diferentes, e também com sistemas de diferentes versões;



ACTIVITY

Um dos componentes mais **comuns** no desenvolvimento Android se chama Activity (ou simplesmente atividade, em português). Podemos entender uma Activity como **uma tela do nosso aplicativo.**

Devemos eleger uma Activity como **principal**, geralmente chamada de **MainActivity**



ACTIVITY FRONT

```
<?xml version="1.0" encoding="utf-8"?>
<TextView xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:text="Hello Kotlin" />
```



ACTIVITY BACK

```
import android.app.Activity
import android.os.Bundle

class MainActivity : Activity() {
        override fun onCreate(savedInstanceState: Bundle?) {
                super.onCreate(savedInstanceState)
                //definindo o conteúdo da tela
                setContentView(R.layout.activity_main)
        }
}
```



ACTIVITY CONTENT

Essa linha pode ser quebrada em três partes, mas é muito simples de entender. Vamos analisá-la de trás para frente. Essa linha de código pressupõe que exista um arquivo xml chamado activity_main , que está em uma pasta especial do Android, chamada layout , e através da variável R conseguimos acessar essa pasta; por isso:

R.layout.activity_main .



CLASSE R

O nome **R** é uma abreviação de **Resources** (Recursos), o que faz todo o sentido. A ideia de chamar a classe **somente de R** é facilitar a vida do desenvolvedor.

- R.layout.activity_main
- R.drawable.background
- R.color.azul



VIEWS – COMPONENTES VISUAIS

As **Views** (visualizações) são todos os **componentes visuais** que podem ser usados na criação de um aplicativo. A classe View é a classe base de qualquer outro componente que podemos usar em uma Activity. **Botões, caixas de texto, caixas de seleção, objetos de imagens etc**., todos são derivados da classe View .



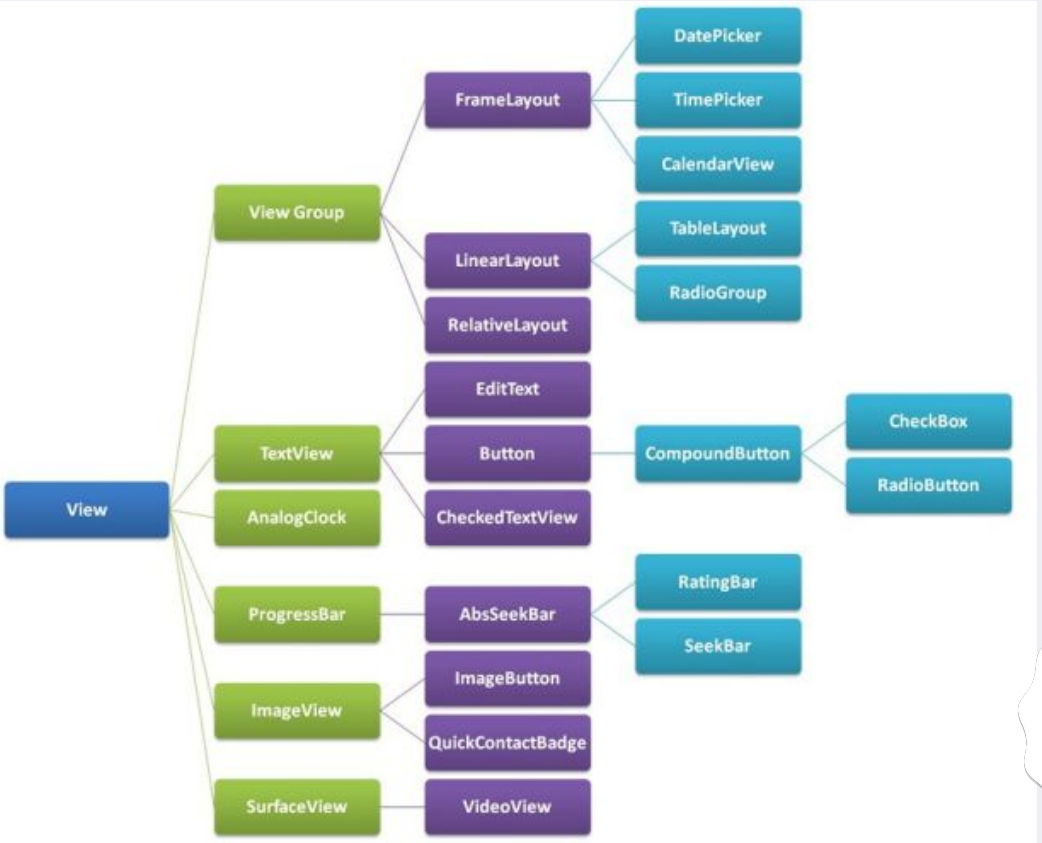


Figura 4.5: Hierarquia da View.



VIEWS – TextView

O TextView é um componente de visualização de texto:

Ele pode ser definido pela tag <TextView />

```
<TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Texto exemplo" />
```



VIEWS – EditText

O EditText , conhecido também como caixa de texto.

Pode ser definido pela tag <EditText />

```
<EditText
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Nome de usuário" />
```



VIEWS – Button

Muito comum é o Button , que cria um botão na tela e pode disparar alguma ação.

Definido pela tag <Button />

```
<Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="cadastrar" />
```



WIDTH & HEIGHT

Em todas as Views, é obrigatório que se definam duas propriedades, layout_width (largura) e layout_height (altura).

- android:layout_width="130dp" \\ Fixo
- android:layout_width="wrap_content" \\Conteúdo
- android:layout_width="match_parent" \\Total



findViewById

Quando precisamos acessar algum elemento que foi definido no arquivo de layout pelo código em Kotlin, precisamos localizar aquele elemento pelo seu ID.



findViewById

```
<Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="login"
        android:id="@+id/button_login" />
```

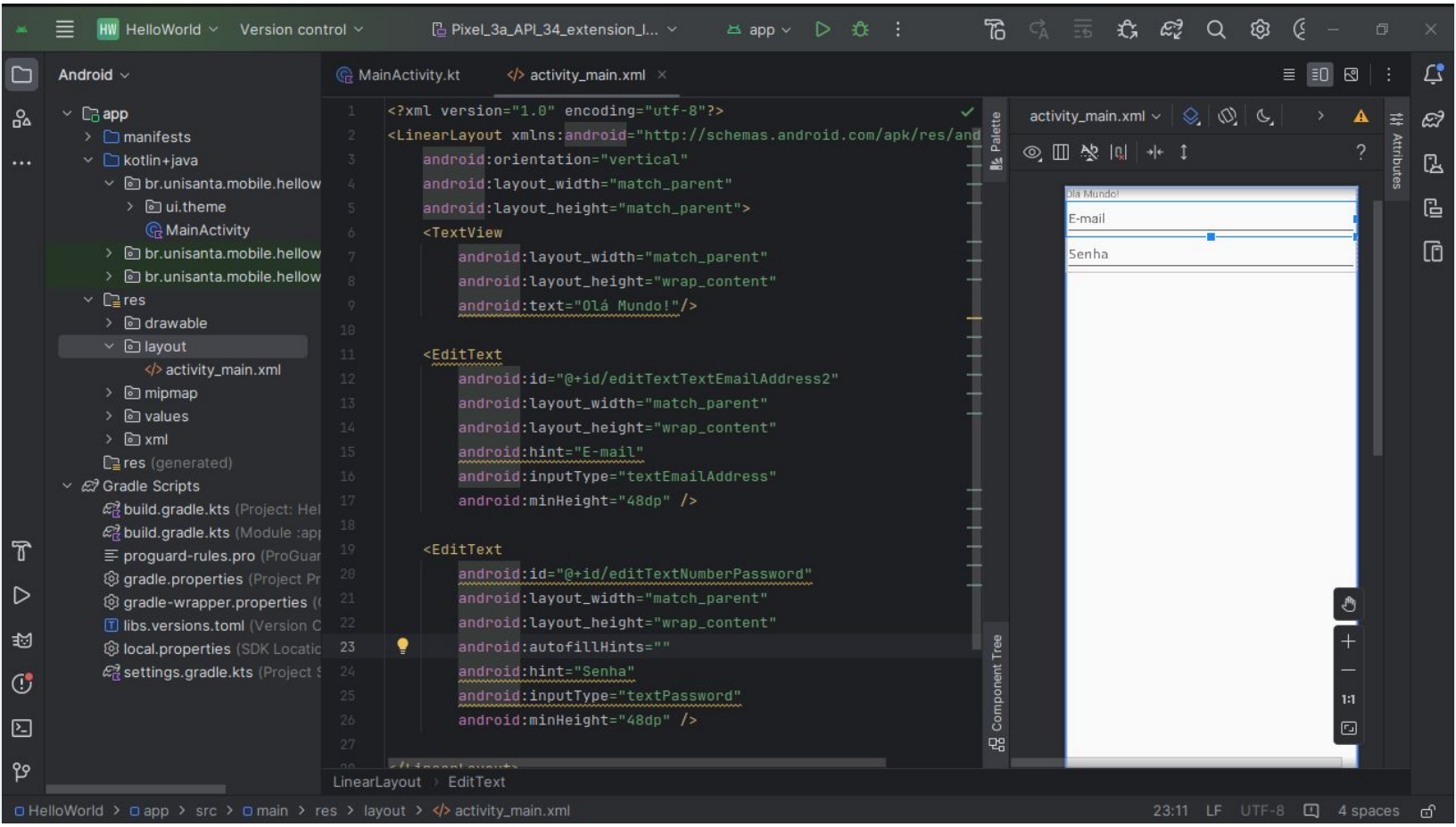
```
val buttonlogin = findViewById<Button>(R.id.button_login)
```



LAYOUT

- Fase De Planejamento
    - Front
    - Back
- CODE || CODE+PREVIEW || GRAFIC





setOnClickListener

Cada componente pode possuir diversos tipos de métodos de listeners. Esses métodos funcionam como verdadeiros ouvintes de algum evento.

```
val btnCalcular = findViewById<Button>(R.id.btn_calcular)

btnCalcular.setOnClickListener {
    //aqui vai o código que será executado quando houver um clique do botão
}
```



DATA BINDING

```
val textValor1 = findViewById<EditText>(R.id.edit_text_valor1)
val textValor2 = findViewById<EditText>(R.id.edit_text_valor2)
val textResultado = findViewById<TextView>(R.id.tex_view_resultado)
val btnCalcular = findViewById<Button>(R.id.btn_calcular)

btnCalcular.setOnClickListener {
        val v1 = textValor1.text.toString().toInt()
        val v2 = textValor2.text.toString().toInt()
        val resp = v1+v2
        textResultado.text = “O Resultado é $resp”
}
```



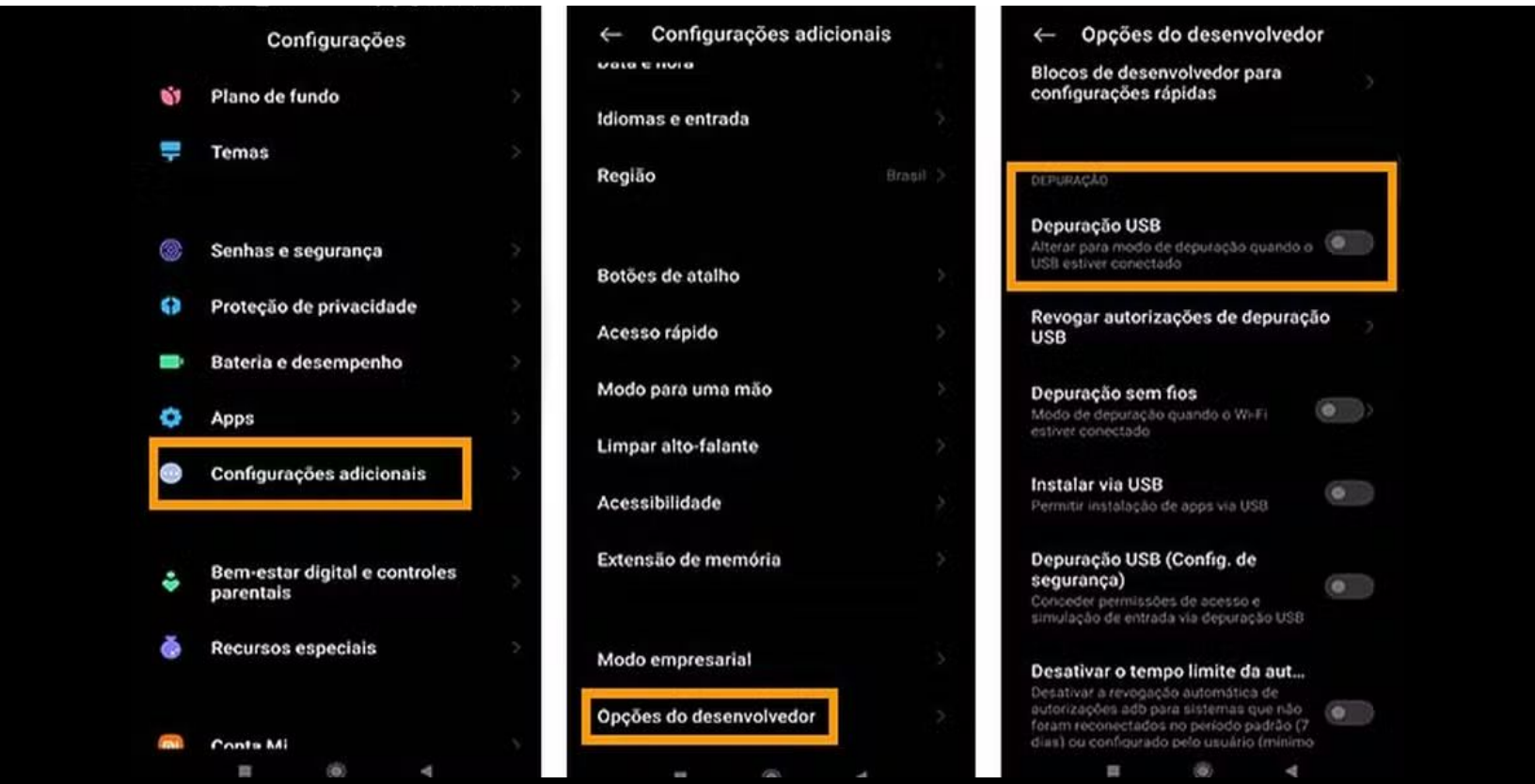
SIMULADOR OU HARDWARE

- Benefícios de se utilizar o emulador
    - Várias Versões
    - Teste em diferentes plataformas
- Benefícios do Celular
    - Uso real
    - Uso de recursos nativos

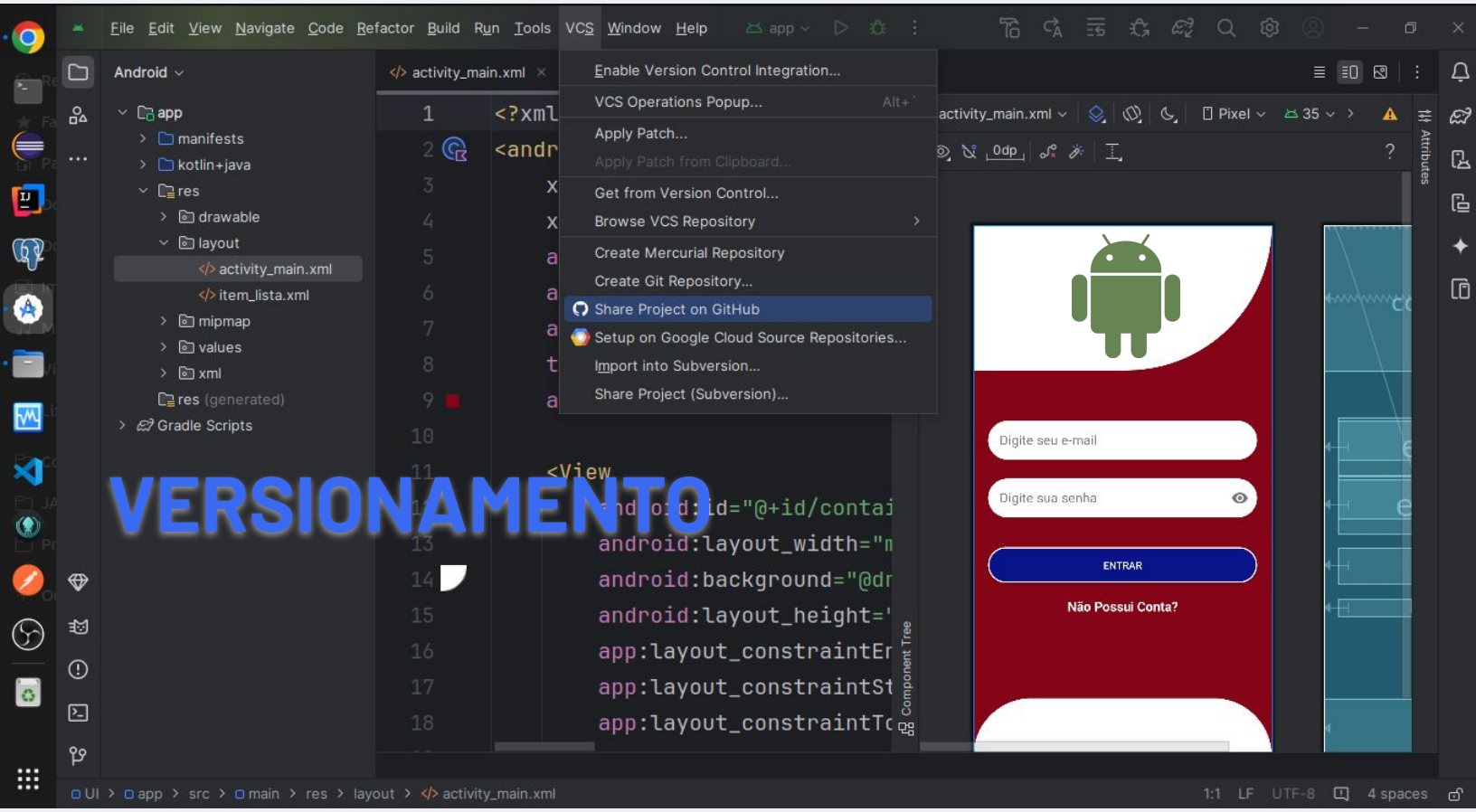


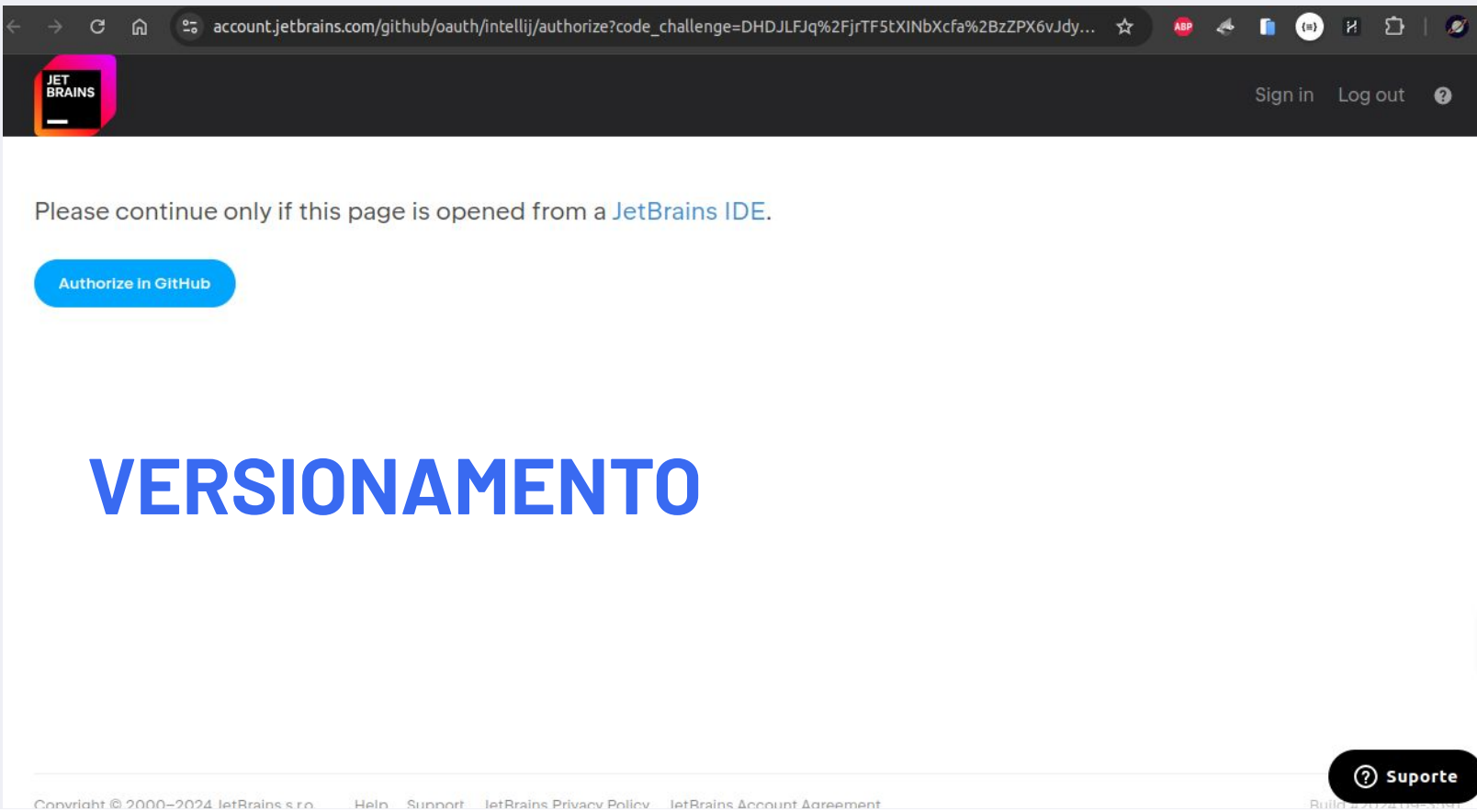
SIMULADOR OU HARDWARE

- Problemas ao se utilizar o emulador
  - Consumo de Hardware do Computador
- Problemas do Celular
  - Insegurança Depuração
  - Desempenho do Celular
  - Versão única



https://www.techtudo.com.br/guia/2023/12/o-que-e-e-como-funciona-a-depuracao-usb-edmobile.ghtml


File Edit View Navigate Code Refactor Build Run Tools VCS Window Help
Android
app
manifests
kotlin+java
res
drawable
layout
activity_main.xml
item_lista.xml
mipmap
values
xml
res (generated)
Gradle Scripts
activity_main.xml
Enable Version Control Integration...
VCS Operations Popup... Alt+`
Apply Patch...
Apply Patch from Clipboard...
Get from Version Control...
Browse VCS Repository
Create Mercurial Repository
Create Git Repository...
Share Project on GitHub
Setup on Google Cloud Source Repositories...
Import into Subversion...
Share Project (Subversion)...
VERSIONAMENTO
Digite seu e-mail
Digite sua senha
ENTRAR
Não Possui Conta?
Component Tree
Attributes
UI > app > src > main > res > layout > activity_main.xml
1:1 LF UTF-8 4 spaces


account.jetbrains.com/github/oauth/intellij/authorize?code_challenge=DHDJLFJq%2FjrTF5tXINbXcfa%2BzZPX6vJdy...
JET BRAINS
Sign in
Log out
Please continue only if this page is opened from a JetBrains IDE.
Authorize in GitHub
VERSIONAMENTO
Copyright © 2000–2024 JetBrains s.r.o.
Help
Support
JetBrains Privacy Policy
JetBrains Account Agreement
Suporte

account.jetbrains.com/github/oauth/intellij/complete

JET BRAINS

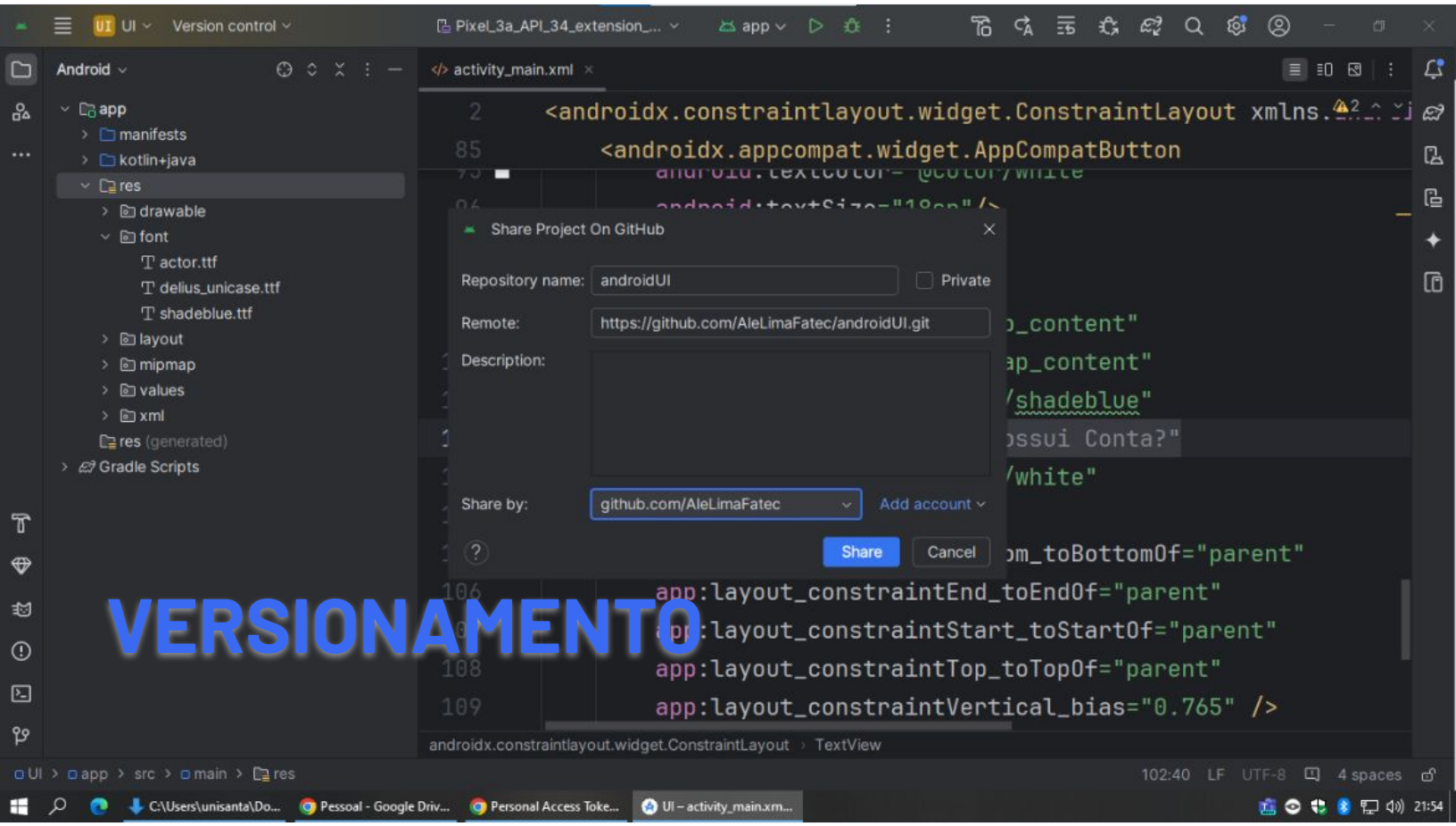
Sign in Log out

You have been successfully authorized in GitHub. You can close the page.

VERSIONAMENTO

Suporte

Copyright © 2000–2024 JetBrains s.r.o. Help Support JetBrains Privacy Policy JetBrains Account Agreement



VERSIONAMENTO

TP2 - KOTLIN

1.Crie um aplicativo que tenha dois campos de texto (EditTexts), um para o nome do usuário e outro para sua idade. Adicione um botão que, ao ser clicado, exiba uma mensagem de boas-vindas na tela, incluindo o nome e a idade do usuário.

2.Crie um App de conversão de temperatura: tenha um campo de texto (EditText) para digitar uma temperatura em Celsius, um botão que realize a conversão para Fahrenheit e um TextView para exibir o resultado da conversão.

